ImageLab v3

Server-Sent Events (SSE) Extension

CMPS4191: Advanced Web Technologies
University of Belize
Andres Hung & Jennessa Sierra
Nov 17, 2026

GitHub: https://github.com/andreshungbz/imagelab

Releases: https://github.com/andreshungbz/imagelab/releases
v2 Branch: https://github.com/andreshungbz/imagelab/tree/v2-long-polling
v1 Branch: https://github.com/andreshungbz/imagelab/tree/week-4

IMAGELAB v3		
	Title	ImageLab v3
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Nov 17, 2026

Table of Contents

Table of Contents **2**
Predictions **3**
Methodology **4**
Results **5**
1) Observation GET (LP) / Stream GET (SSE) Count by Condition & Mode 6
1X) Observation GET (LP) / Job Events (SSE) Count by Condition & Mode 7
2) Completion-Detection Delay by Condition & Mode 8
3) Observation Database Reads Count by Condition & Mode 9
Interpretation **10**
Limitations **11**
Recommendation **11**
Answered Questions **12**
Student Explanation 12
In-class Presentation Questions 13

IMAGELAB v3		
	Title	ImageLab v3
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Nov 17, 2026

Predictions

Under all conditions, no artificial image processing or worker-start delays are assumed.

Condition	Controlled Workload	Question	Long Poll Prediction	SSE Prediction
A. Quiet Interval	Keep a job active for 30 seconds with no client-visible progress, then perform real processing.	How does each method behave when the job is active but has no update to send?	The first long poll will time out due to the default 20s wait timer. The first request will complete with a 204 No Content response, then another long poll is sent.	The first and single SSE request will be retained throughout. The heartbeat will be seen after 20s, but it will not be in another request.
B. Real Progress	Process the real thumbnail, preview, and display variants. Publish progress only after each variant and its metadata are stored.	How quickly does each method deliver real progress and terminal state?	Long polling will be very quick at delivering progress updates, although there will probably be some overhead because the browser needs to construct the next long-poll request with each update.	SSE will be very quick as well, and probably quicker than long polling, since it doesn't require the overhead of constructing additional requests.
C. Frequent Updates	Use a labeled 10-second simulated phase with updates every 200 ms, then perform real processing.	How do update frequency, observed versions, and delivery behavior differ?	Update frequency will be high, with each observed version seen up to 55 times if none are skipped due to timing variance (50 during the simulated phase and 5 during real processing). There will be up to 55 individual long-poll requests per update.	The update frequency and observed versions will be the same as with long polling. The delivery behavior will differ: there will still be only one SSE request. In that event stream, there will be up to 55 updates, though.

IMAGELAB v3		
	Title	ImageLab v3
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Nov 17, 2026

Methodology

Version 3 of ImageLab implemented an additional asynchronous application programming interface (API): Server-Sent Events (SSE). To implement this, an additional handler function was created in the backend Go API server. Using a channel-based notification system, the worker notified the handlers whenever it wrote a job update to the database. The SSE handler waits for an update before sending a **job-update** event. Such an event flushed the Go buffers. When there were no updates, the handler sent a heartbeat (set to 20s). Lastly, on cancellation, the handler stops the SSE stream while the worker continues working.

On the frontend, the EventSource object was used to establish an SSE stream with the API server. It was configured to recover on cancellation and network disconnection. User interface (UI) elements were implemented to add the SSE option. As in Version 2, mode changes during processing were accounted for. To demonstrate simultaneous observation of a single job in two different browser tabs, an additional input section was added to the jobs section.

For measurements, Conditions A, B, and C2 were reused from Version 2, with C2 being renamed to C in Version 3. Measurements were conducted with the same device and image file as used in Version 2 measurements. Three (3) runs were conducted for each condition, and tables and charts were constructed from the raw data using a helper consolidation Python script. As with Version 2 measurements, the browser logs, server logs, and HTTP Archive (.har) files were saved. For SSE tests, the event request logs were also recorded, as they aren’t saved in the .har files. All tables, charts, and raw logs were saved to the project repository.

In addition to the main requirements, extensive refactors, documentation, and bug fixes were completed.

IMAGELAB v3		
	Title	ImageLab v3
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Nov 17, 2026

Results

Refer to the following external links for the raw logs and run-by-run measurement table. The raw measurements table is presented in the Google Sheets form to make the results easier to view. It is not shown here because there are too many columns to fit within a reasonable space.

Raw Logs/Files (GitHub): https://github.com/andreshungbz/imagelab/tree/v3-sse/docs/v3.0.0/experiments

Run-by-Run Measurement Table (Google Sheets): v3-tables

The following list describes the summary tables and corresponding charts created, noting which were required and which are extra. Long polling is indicated by “LP,” and Server-Sent Events is indicated by “SSE.”

- Observation GET (LP) / Stream GET (SSE) Count by Condition & Mode (Required)
- Observation GET (LP) / Job Events (SSE) Count by Condition & Mode (Extra)
- Completion-Detection Delay by Condition & Mode (Required)
- Observation Database Reads Count by Condition & Mode (Extra)

The following pages show each table alongside the chart.

IMAGELAB v3		
	Title	ImageLab v3
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Nov 17, 2026

1) Observation GET (LP) / Stream GET (SSE) Count by Condition & Mode

Condition	Mode	Observation GET (LP) / Stream GET (SSE) Count			
		Minimum	Maximum	Median	Average
A	Long Polling	6	6	6	6.0
A	SSE	1	1	1	1.0
B	Long Polling	4	5	4	4.3
B	SSE	1	1	1	1.0
C	Long Polling	53	55	54	54.0
C	SSE	1	1	1	1.0

Table 1: Summary Table for Observation GET (LP) / Stream GET (SSE) Count by Condition & Mode.

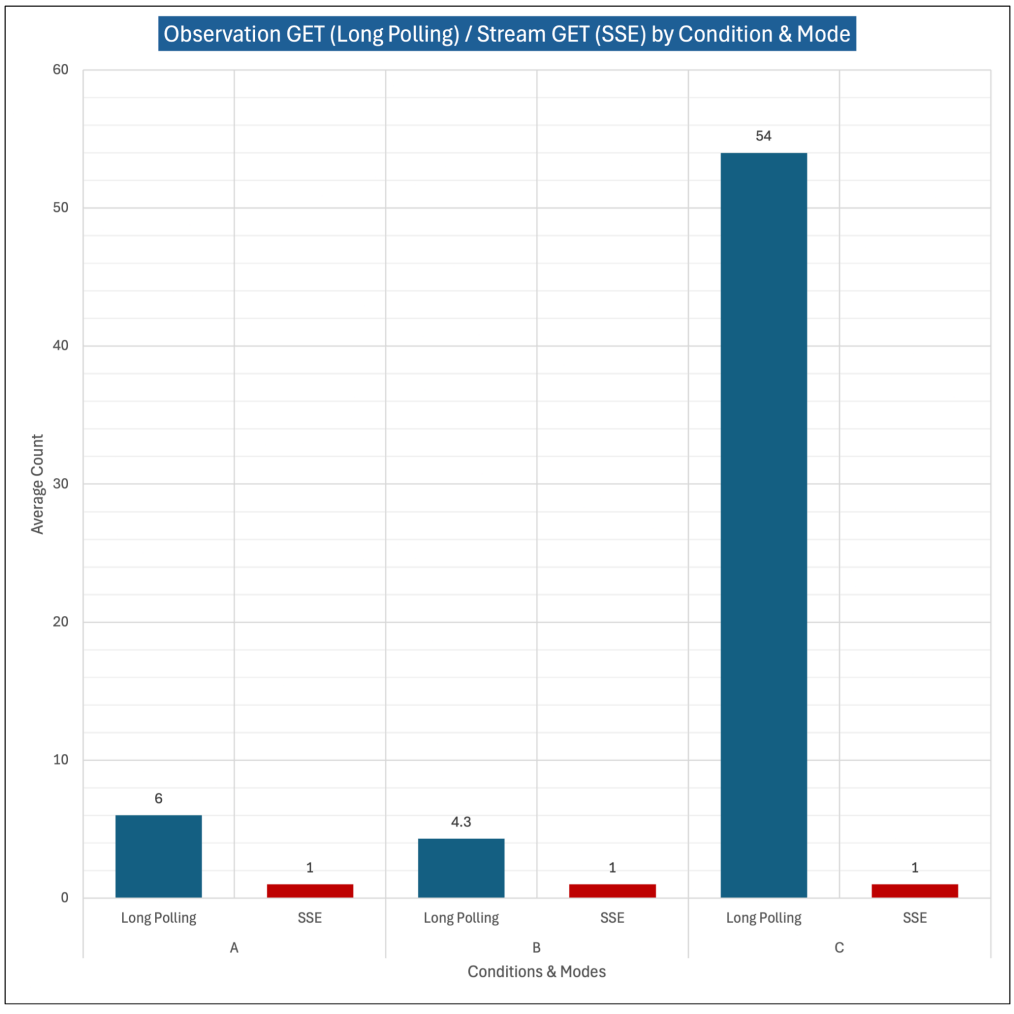


Figure 1: Chart for Observation GET (LP) / Stream GET (SSE) Count by Condition & Mode.

IMAGELAB v3		
	Title	ImageLab v3
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Nov 17, 2026

1X) Observation GET (LP) / Job Events (SSE) Count by Condition & Mode

Condition	Mode	Observation GET (LP) / Job Events (SSE) Count			
		Minimum	Maximum	Median	Average
A	Long Polling	6	6	6	6.0
A	SSE	5	5	5	5.0
B	Long Polling	4	5	4	4.3
B	SSE	5	5	5	5.0
C	Long Polling	53	55	54	54.0
C	SSE	55	55	55	55.0

Table 2: Summary Table for Observation GET (LP) / Job Events (SSE) Count by Condition & Mode.

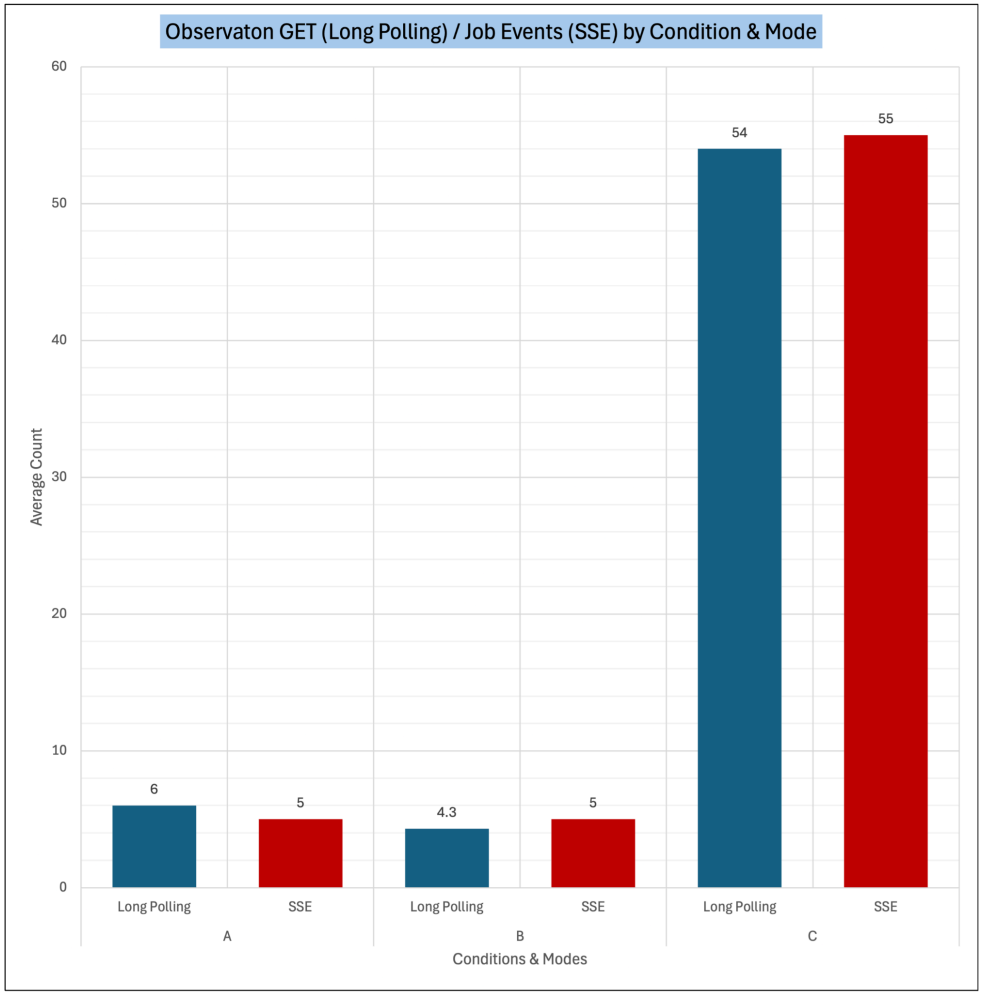


Figure 2: Chart for Observation GET (LP) / Job Events (SSE) Count by Condition & Mode.

IMAGELAB v3		
	Title	ImageLab v3
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Nov 17, 2026

2) Completion-Detection Delay by Condition & Mode

Condition	Mode	Completion-Detection Delay (ms)			
		Minimum	Maximum	Median	Average
A	Long Polling	103	126	124	117.7
A	SSE	1	4	3	2.7
B	Long Polling	31	71	40	47.3
B	SSE	3	5	4	4.0
C	Long Polling	42	115	95	84.0
C	SSE	4	4	4	4.0

Table 3: Summary Table for Completion-Detection Delay by Condition & Mode.

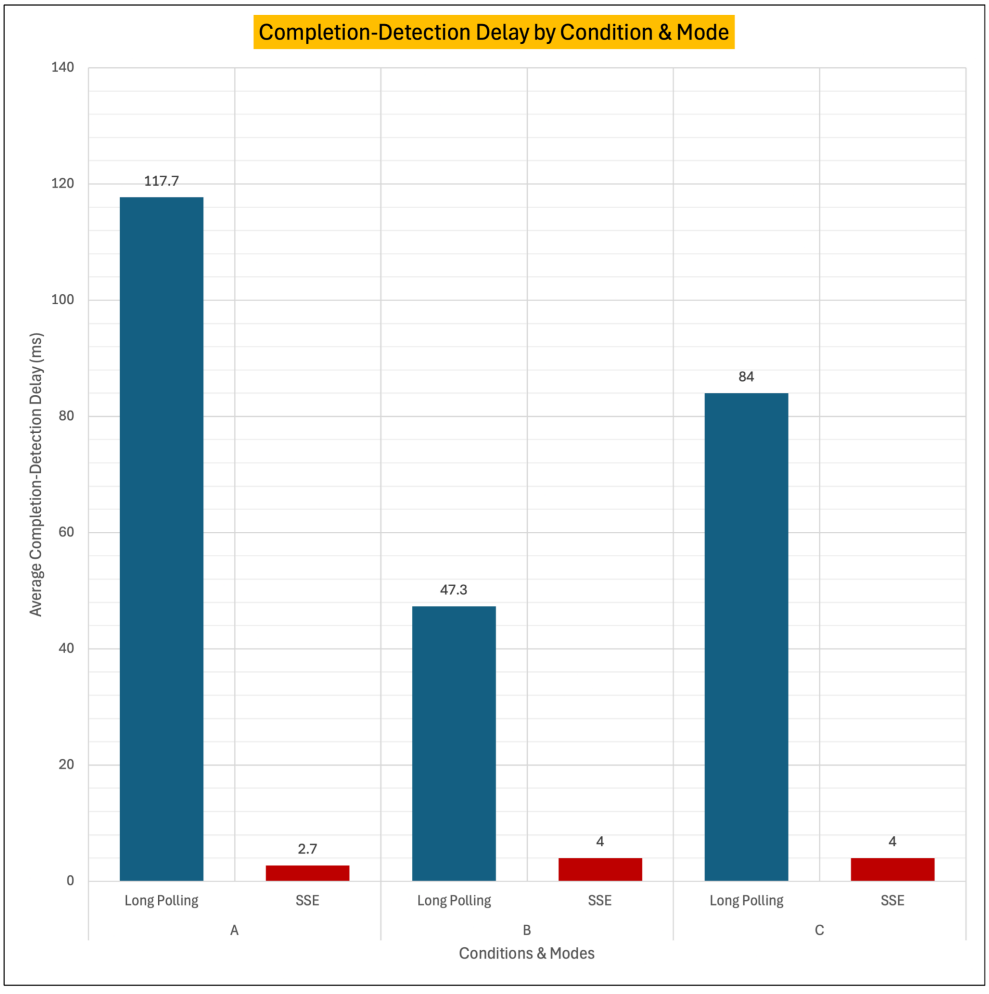


Figure 3: Chart for Completion-Detection Delay by Condition & Mode.

IMAGELAB v3		
	Title	ImageLab v3
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Nov 17, 2026

3) Observation Database Reads Count by Condition & Mode

Condition	Mode	Observation Database Reads Count			
		Minimum	Maximum	Median	Average
A	Long Polling	160	160	160	160.0
A	SSE	5	5	5	5.0
B	Long Polling	8	9	8	8.3
B	SSE	5	5	5	5.0
C	Long Polling	105	108	107	106.7
C	SSE	55	55	55	55.0

Table 4: Summary Table for Observation Database Reads Count by Condition & Mode.

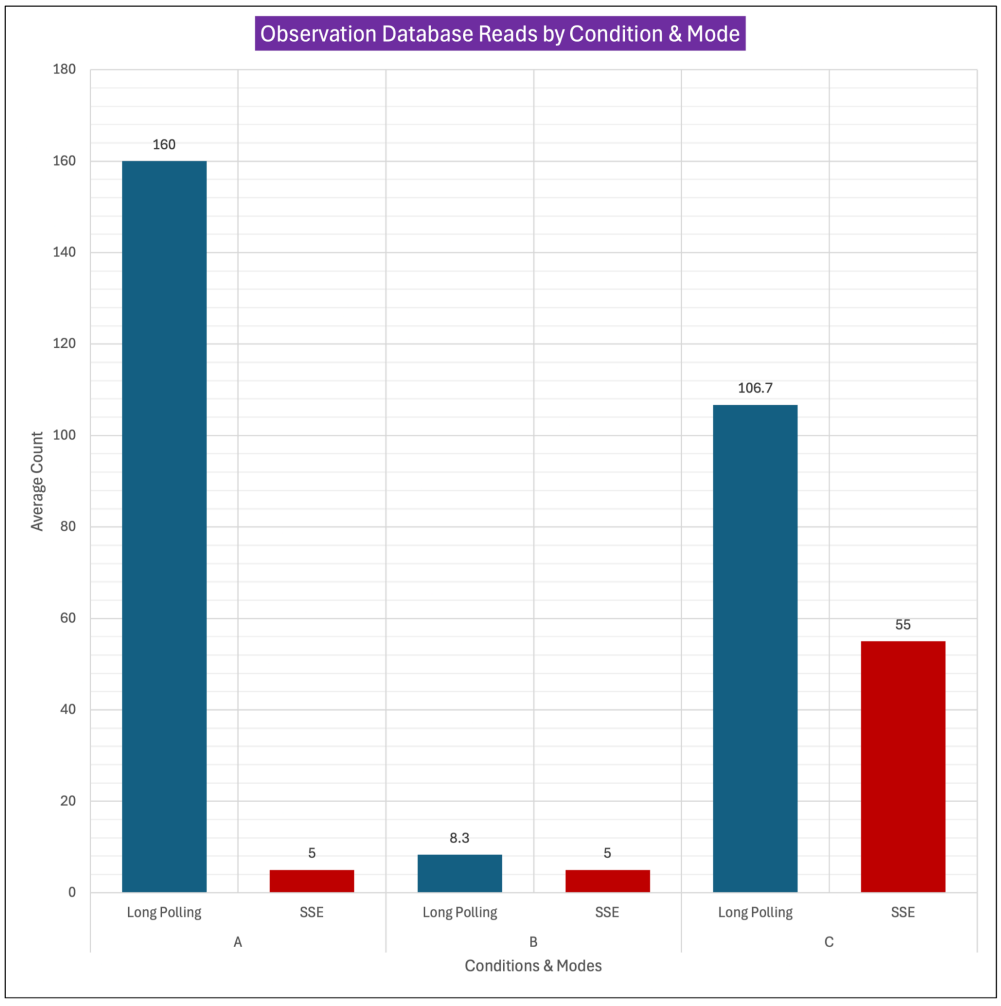


Figure 4: Chart for Observation Database Reads Count by Condition & Mode.

IMAGELAB v3		
	Title	ImageLab v3
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Nov 17, 2026

Interpretation

Long Polling uses observation GET requests, while SSE uses stream GET requests. Comparing the counts of those requests in Figure 1, SSE was more effective, as it required only 1 request to manage job updates for both real processing and simulated updates. Condition C, with simulated updates, showed that Long Polling required a request for each update. However, counting the SSE Stream GET requests doesn't tell the entire story. In Figure 2, which compares Long Polling observation GET requests with SSE job events, the counts are essentially the same across the modes. For every job update, real or simulated, that information is sent back to the client through the network. Long Polling does this by sending another long poll, and SSE sends it through the initially created request.

Comparing completion-detection delays between Long Polling and SSE, SSE clearly outperforms it across all conditions, with delays in the single-digit millisecond range (see Figure 3). Long Polling delays were still shorter than short polling, but these higher delays can be attributed to browser overhead for tearing down and re-establishing long-poll connections. Since SSE uses a single connection, the client can process the terminal commit more quickly. The variance in Long Polling delays between conditions can be attributed to browser overhead and timing variance due to the timer/ticker-based implementation. Since the ticker is configured to fire every 200ms, the delay is bounded around that value. There was no latency configured for any of the conditions, so latency was of minimal impact.

A notable comparison between the two modes across all conditions was the number of observation database reads each made (see Figure 4). SSE consistently made fewer reads than Long Polling and consistently returned the exact number of job updates. The main reason Long Polling makes more reads is the timer/ticker implementation. Since SSE uses a channel-based approach, reads are only counted when handlers are notified of an update. On the other hand, Long Polling with a constant 200ms ticker results in reads that don't have any updates. If Long Polling were to be implemented using a channel-based approach, it should match the SSE results in this regard.

Processing duration stayed consistent between both Long Polling and SSE, aligning with the delays set by the conditions. Actual image processing remained at around 550ms to 600ms. SSE stream duration also matched the conditions, adding about 100ms to the processing duration. Since each measurement was conducted with only one worker and one browser tab, the peak SSE streams were always 1. Peak streams typically increase when multiple browser tabs observe the same job using SSE. Notably, SSE never had any version gaps, whereas Long Polling sometimes did. This is attributed to the timing variance in how each long poll is created, sent, and processed. Because SSE sends the next job update event over the same single connection, missing an update is not possible unless the data is lost over the network.

The results matched the predictions previously made. The only adjustment that could be made would be the degree to which SSE was faster at showing the terminal state (completion-detection delay). While these results show that SSE is better than Long Polling in terms of latency, bandwidth efficiency, and continuous streaming, SSE is stateful and unidirectional from server to client. Long Polling is easier to load-balance given its statelessness and is semi-duplex, with requests and responses exchanged.

IMAGELAB v3		
	Title	ImageLab v3
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Nov 17, 2026

Limitations

- The timer/ticker implementation for Long Polling could have affected the comparison of results, whereas a channel-based implementation would reveal more differences with SSE, which uses that approach.
- Variable device CPU load may have influenced processing on the server and client, which in turn affected the recorded times.
- Localhost measurements do not account for network latency.
- Multiple images or multiple job observations were not tested.

Recommendation

SSE greatly reduces completion-detection delay compared to Long Polling and generally sends fewer requests. Both are still viable asynchronous APIs depending on the situation. Long Polling can be preferable when bidirectional communication is required between the client and server (SSE is unidirectional from the server to the client). Long Polling is preferred for scenarios involving large batch submissions, where the client isn't just receiving data from the server. ImageLab is a great scenario for SSE, since the client primarily receives periodic job status notifications from the server.

A channel-based implementation of Long Polling remains a recommendation from Version 2. For ImageLab as a whole, the WebSockets asynchronous API could be explored. Additional image variants can be added, and multiple workers can be implemented to explore ways to scale the application.

IMAGELAB v3		
	Title	ImageLab v3
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Nov 17, 2026

Answered Questions

Student Explanation

1. **What does the GET open, and what does Flush do?**

The SSE stream GET request opens a persistent connection between the client and server using the EventSource object. The server sends multiple updates over that same connection. Flush enables this by emptying the Go HTTP buffers so that the client immediately receives updates.

2. **How do workers, PostgreSQL versions, notifications, SSE handlers, and browser listeners fit together?**

The PostgreSQL database serves as the authoritative source for jobs, tracking versions as job statuses change or image variants become available. Notifications (channel-based) activated by the worker communicate updates to subscribed SSE handlers, which in turn send an event to the browser. Lastly, the browser listens for these events to render job updates appropriately.

3. **What does a disconnect change, and what remains active?**

When the SSE stream disconnects for whatever reason, the worker and image processing remain active. A new stream can be created, and it will receive the event with the latest update.

4. **Why does this specification recover a snapshot instead of replaying all missed events?**

This specification recovers a snapshot, since replaying all missed events could cause the client to wait indefinitely for an update that was lost along the way. By recovering the latest snapshot, notifications are safely coalesced. Replaying all missed events would require additional implementation and event persistence.

5. **What do the tables and graphs show, and what limitations remain?**

Generally, the tables and graphs show that SSE performs better than Long Polling. It sends fewer requests and has a shorter completion-detection delay. Read the Interpretation section for a more in-depth analysis. Limitations remain regarding real-network measurement testing, multiple workers, simultaneous loads, and the implementation of equal notification across both modes (see Limitations).

IMAGELAB v3		
	Title	ImageLab v3
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Nov 17, 2026

In-class Presentation Questions

1. **What does the server keep open, and what does Flush do?**

The server keeps the SSE stream connection open, sending events unidirectionally to the client. Flush, called by the server, allows the server to decide when the client receives an update; otherwise, response data is buffered.

2. **How does a committed job version become an SSE event for every connected tab?**

The backend uses a channel-based notification implementation in which a map is maintained for each SSE handler that subscribes to notifications for a particular job. Upon notification of a committed job version, the SSE's independent channels each receive a value that is read in a select statement. Then an SSE event is sent to every connected tab.

3. **What happens when a connection drops while the worker is still running?**

Because the worker is independent of the SSE handler and connection, it continues to run.

4. **Why can a reconnecting client receive the latest snapshot without replaying every intermediate version?**

The SSE handler is implemented to read the latest job state from the database, on first subscription or as part of a notification update. The PostgreSQL database maintains the authoritative state and contains all the information required to interpret any intermediate versions that might have been skipped.

5. **What did the graphs show that was harder to see in the tables, and what do the results not establish?**

The graphs more clearly showed the extent to which SSE outperformed Long Polling across request count, completion-detection delay, and observation database reads. The results don't establish that SSE is always better than Long Polling, though. SSE's unidirectionality and stateful nature make it suitable for certain applications, but not for all. SSE would work better for live data feeds and notifications, such as ImageLab, and Long Polling would work better for large batch submissions, where the client doesn't solely receive data from the server.